

1>---- answer

```
from decimal import Decimal
from flask import request, jsonify
from sqlalchemy.exc import IntegrityError

@app.route('/api/products', methods=["POST"])
def create_product():
    data = request.json

    try:
        # Validation
        if not data.get('name') or not data.get('sku'):
            return jsonify({"error": "Name and SKU are required"}), 400

        # Check SKU uniqueness
        existing = Product.query.filter_by(sku=data['sku']).first()
        if existing:
            return jsonify({"error": "SKU already exists"}), 400

        # Start transaction
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=Decimal(str(data.get('price', 0)))
        )

        db.session.add(product)
        db.session.flush() # get product.id before commit

        # Inventory creation (multi-warehouse support)
        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=data.get('initial_quantity', 0)
        )

        db.session.add(inventory)

        db.session.commit()

    return jsonify({
        "message": "Product created",
        "product_id": product.id
    })
```

```
}}, 201
```

```
except IntegrityError:  
    db.session.rollback()  
    return jsonify({"error": "Database error"}), 500
```

```
except Exception as e:  
    db.session.rollback()  
    return jsonify({"error": str(e)}), 500
```

2> answer

Companies

id (PK)

name

Warehouses

id (PK)

company_id (FK)

name

location

Products

id (PK)

name

sku (UNIQUE)

price

product_type

Inventory

id (PK)

product_id (FK)

warehouse_id (FK)

quantity

Inventory_Logs

```
-----  
id (PK)  
product_id  
warehouse_id  
change  
created_at
```

Suppliers

```
-----  
id (PK)  
name  
contact_email
```

Product_Suppliers

```
-----  
product_id (FK)  
supplier_id (FK)
```

Bundles

```
-----  
id (PK)  
bundle_product_id  
child_product_id  
quantity
```

3> answer

```
@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])  
def low_stock_alerts(company_id):  
    alerts = []
```

```
    inventories = db.session.query(Inventory)\  
        .join(Product)\  
        .join(Warehouse)\  
        .filter(Warehouse.company_id == company_id)\  
        .all()
```

```
    for inv in inventories:  
        product = inv.product
```

```
        # Assume threshold based on product type  
        threshold = 20 if product.product_type == "standard" else 10
```

```

# Check recent sales (last 30 days)
recent_sales = Sale.query.filter(
    Sale.product_id == product.id,
    Sale.created_at >= datetime.utcnow() - timedelta(days=30)
).count()

if inv.quantity < threshold and recent_sales > 0:

    supplier = ProductSupplier.query\
        .filter_by(product_id=product.id)\
        .first()

    alerts.append({
        "product_id": product.id,
        "product_name": product.name,
        "sku": product.sku,
        "warehouse_id": inv.warehouse_id,
        "warehouse_name": inv.warehouse.name,
        "current_stock": inv.quantity,
        "threshold": threshold,
        "days_until_stockout": 10, # assumed logic
        "supplier": {
            "id": supplier.supplier.id if supplier else None,
            "name": supplier.supplier.name if supplier else None,
            "contact_email": supplier.supplier.contact_email if supplier else None
        }
    })

return {
    "alerts": alerts,
    "total_alerts": len(alerts)
}

```